

```
/****** ReactJs Enviornment up with Windows *****/
/*
```

1. Install NodeJs and NPM.

Ans :- 1. Install nodeJs

2. VS code editor.

3. Install CRA (Create React App) app.

Ans :- npx create-react-app project_name (blog).

4. Interview question.

Steps to install and used the ReactJs App.

1. Install nodeJs

2. Check the version of nodeJs in command line :- node -v

3. Check the NPM version :- npm -v

4. Make a folder using cmd :- mkdir folder_name (react-tut)

5. Goto the react-tut folder using cmd :- cd react-tut

6. Check the npx in the cmd :- npx -v (This is install with the nodeJs)

7. Create a react project using cmd :- npx create-react-app project_name (blog)

8. Goto the blog folder and open inside the Visual studio using cmd :- code .

9. Start the react project with cmd :- npm start

Url :- http://127.0.0.1:5500/index.html

After that react project :-

Url :- Local: http://localhost:3000

On Your Network: http://192.168.42.245:3000

*/

```
/****** ReactJs write First code *****/
```

```
/*
```

1. Write first code.

Ans :- When you write a first. you will write first code in the App.js file.

First file is load in the react is index.js file.

2. Make a new file.

Ans :- Create a first file inside the src folder. first letter of the file always be capital.

Users.js.

3. Interview question.

Q :- Alternative of NPM?

Ans :- Yarn is alternative of NPM. Yarn is a product of facebook.

Q :- Current version of ReactJs?

Ans :- Using CMD command check the version of ReactJS :- npm view react version

In side the package.js file you will also check the current version of ReactJs.

*/

```
/****** ReactJs App Structured *****/
```

```
/*
```

1. Understand File use and structure.

Ans :- 1. App.css :- File is basically used for styling.

2. App.js :- We have start write from App.js file.

3. App.test.js :- In this write the unit test cases. For example :- Developer write unit test case. Developer run the test case using CMD. The load of testing team is less and We know the where is some Bug inside the app.

This is a advanced level thinks.

4. Index.css :- File is basically used for styling. You have make one or more file for styling.

5. Index.js :- This is file is enter point of ReactJs.

6. reportWebVitals.js :- This file is get the report of application performance. This file embbed inside the index.js file.

7. setupTests.js :- All test case of App.test.js file is setup start from setupTests.js file.

2. Understand Folder use and structure.

Ans :- 1. Public folder :- Public folder stored the details of HTML related that not related with ReactJs

For example :- Favicon.ioc, index.html. All the work of ReactJs run inside the index.html file div. <div id="root"></div>

2. Inside the (src) folder we have worked of ReactJs related.

3. manifest.json :- manifest.json also called meta file. This is stored the some icon or display color theme etc.

This used when developed the basic level app (Prograsive app).

4. robots.txt :- Protect form google serach engine.

5. node_module :- All the package are stored inside this folder. All package code are avaiable inside this folder.

6. build :- This folder is created when we make a build.

3. Interview question.

Q :- Most important file inside ReactJs?

Ans :- Package.json file is most important file.

Q :- Why we not push the node_module folder in Git?

Ans :- Because it's too heavy and include the node_module it's very have for git repository.

Q :- How get the node_module when the other developer is working on the project?

Ans :- npm install :- This command is install all the node_module inside the reactjs project. If we have already installed it's check the modules.

*/

/***** Package.json file in Details *****/

/*

1. What is package.json file.

Ans :- Package.json file is most important file. It's stored all the details of projects.

2. How to add new NPM package.

Ans :- find the react npm package. Using the command line and past the package install command.

like :- npm i react-validation

Second method to install the package :- Goto npm package >>

version >> open the version and copy and past inside the

package.json file after that run the npm install command. Package are installed automatically.

3. Remove NPM package.

Ans :- 1. npm uninstall package_name

2. Remove the package form package.json file and npm install

4. Custom commands.

Q :- Suppose npm is start. But we don't start npm with start. I want to start npm with other name?

Ans :- Goto package.json file >> "start": "react-scripts start", and replace the "start" with custom_name.

After that goto CMD and run the this command :- npm run gurjeet (custome_name)

4. Interview question

Q :- How many type of component in ReactJs?

Ans :-

*/

/****** Component in ReactJS *****/

/*

1. What is component.

Ans :- Piece of code that can reuse. such as function. but more powerful than function. Because his own lifecycle, state, props, own html.

Example :- Header and footer.

2. Types of component.

Ans :- There is basically two types of components.

1. Functional component

2. Class component.

Some other component :-

1. HOC (High order component)

2. Pure component.

3. Controlled component.

4. Uncontrolled component.

3. Example of component.

Ans :- We don't need a different file for create component.

Example :- User.js file

// Functional Component

function User(){

return <h1>User component</h1>

}

// export the component with export default component_name. This

is standard way to export

//export default User;

// Second way to export. Use the export before the function.

// export function User(){

// return <h1>User component</h1>

// }

// Inside the file where you want to import used the : import

{component_name} from './User'

// Example :- import {User} from './User'

App.js file

// import User from './User';

// import {User} from './User';

4. Interview question.

Q :- Can we used the component inside the another component?

Ans :- Yes, We can used the component inside the another component.

Example :-

function App() {

function Apple(){

return (<div>Component inside the another

component.</div>);

}

return (

<div className="App">

<h1>Hello App File</h1>

<User />

<User_class_compnt />

<Apple /> // Two way to call component 1. <Apple /> 2. {Apple}

```

        </div>
    );
}
*/

```

```

/***** Class Component *****/
/*

```

1. What is Component.

Ans :- Component is a piece of code that we used again and again.

2. Make class component.

Ans :- // First method of import component

```

import React from 'react';
class User_class_compnt extends React.Component{
}

```

Functional component is used mostly and class components used in react like 20%

Don't forget render() in class component.

Some difference in class component like props calling, state define etc.

```

import React,{Component} from 'react';
export default class User_class_compnt extends Component{
  render(){
    return (
      <h1>Hello Class Component</h1>
    );
  }
}

```

3. Interview Question.

Q :- Can, We used the Reactjs and Angular in same project.

Ans :- Yes, ReactJS is library and library is used anywhere.

```

*/

```

```

/***** JSX With ReactJS *****/
/*

```

1. Javascript xml (JSX).

Ans :- Why the requirement of JSX.

:- In the past we don't used the HTML and javascript together. We write both in different differently. With help of JSX we used both together. Can, We don't used React without JSX.

Know, We used the React without JSX. Without JSX it's very complicated.

2. Write HTML and Javascript together .

```

Ans :- function User_JSX(){
  return (<div>
    <h1>Hello JSX</h1>
    {10 + 20}
  </div>);
}

```

```

// Without JSX
import React from 'react';
function User_JSX(){
  return React.createElement("h1", null, "HTML without JSX");
}

```

```

export default User_JSX;

```

3. Use like HTML or XML tags.

Ans :- <User_JSX />.

4. React without JSX.

5. Interview question.

Q :- Can, We used the JSX without ReactJS?

Ans :- Yes, JSX is file extension. We can used JSX without ReactJS.

*/

/****** Click Event and Function *****/

/*

1. How to make click event.

Ans :- <button onClick={() => alert("hello")}>Click me</button>

2. Call function in the button click.

Ans :- <button onClick={Apple}>Click me</button>

3. Variable in function component.

Used this code inside the App.js file.

Ans :-

```
let data = "Gurjeet Singh";
```

```
function Apple(){
```

```
  data = "Harashdeep Singh";
```

```
  return alert(data);
```

```
}
```

```
return (
```

```
  <div className="App">
```

```
    <h1>Hello App File</h1>
```

```
    <h3>{data}</h3> // When we change the variable value but it's  
showing pervious value. That's why we used state and props for handle  
this type situation.
```

```
    This alert is automatically called.
```

```
    <button onClick={alert("hello")}>Click me</button>
```

```
    // This also called automatically.
```

```
    <button onClick={Apple()}>Click me</button>
```

```
    // To handlie this using the function without round bracket "()".
```

Like this.

```
    <button onClick={Apple}>Click me</button>
```

```
    // You want to used alert inside the click event. Then you used  
anomynous function inside the event.
```

```
    Like this :- <button onClick={() => alert("hello")}>Click  
me</button>
```

```
    If you want to used a function inside the event. Then used the  
anomynous function. Beacuse this anomynous function not called  
automatically.
```

```
    <button onClick={() => Apple()}>Click me</button>
```

```
  </div>
```

```
);
```

```
}
```

4. Interview question.

Q :- Why value is not changed when the variable value is changed?

Ans :- When we update value inside the variable our component is not
rerender.

*/

/****** State in Functional Component *****/

/*

1. What is State.

Ans :- State is basically object. Like variable we stored data. In the state we also stored data.

2. Use of state.

Ans :- Why used state and not used variable :-

Variable :- When we used the variable our component is not updated.

State :- When we used the state. Like onclick() it's automatically understand the some change in state and print it.

In the variable case this is not happened. State only exists in ReactJS. But variable is in Javascript, Angular etc.

```
function App() {  
  let data = "Gurjeet";  
  function updateData(){  
    data = "Singh";  
    alert(data);  
  }  
}
```

```
return (
```

```
<div className="App">
```

```
<h1>{data}</h1>
```

```
<button onClick={updateData}>Update data</button>
```

Variable value is not change. The reason is ReactJS working method. ReactJS is not working like that.

ReactJS update the data when his State or Prop are changed or Rerender state or prop value.

ReactJS understand State and Prop. His not understand the variable.

```
</div>
```

```
);
```

```
}
```

3. Define state.

Ans :- Import the state :- `import { useState } from 'react';`

```
let [data, setData] = useState("Gurjeet");
```

4. Update state with a button click.

App.js file.

Ans :-

```
function App() {
```

```
// let [data, setData] = useState("Gurjeet");
```

```
let [data, setData] = useState(0);
```

```
function updateData(){
```

```
//   setData("Singh");
```

```
setData(data+1);
```

```
//alert(data);
```

```
}
```

```
return (
```

```
<div className="App">
```

```
<h1>{data}</h1>
```

```
<button onClick={updateData}>Update data</button>
```

```
</div>
```

```
);
```

```
}
```

5. How state work.

6. Interview question.

Q :- Can, We used the component state outside the component?

Ans :- No, ReactJs ask don't use. But some way to used component state outside the component. But this is not authenticate way.

Q :- Tell me the State is public or private?

Ans :- State are public.

*/

```
/*
***** State in Class Component *****
*/
```

1. Make class component.

Ans :- import React,{Component} from 'react';

2. What is state.

Ans :- State is data container. Where we stored the data.

3. Define state.

```
Ans :- constructor(){
    super(); // This a javascript function. When use the "this" or
when we extend the class because the parent constructor is called.
    this.state = {
        //data : "Class State Component"
        data : 0
    }
}
```

4. Update State with button click

```
Ans :- class App extends Component{
    constructor(){
        super(); // This a javascript function. When use the "this" or
when we extend the class because the parent constructor is called.
        this.state = {
            //data : "Class State Component"
            data : 0
        }
    }
    apple(){
        //alert("apple");
        //this.setState({data:"Class Component"});
        this.setState({data:this.state.data+1});
    }
    // In the class we defined the constructor() or super() if you
want used the "this" keywords.
    render(){
        return (
            <div className="App">
                <h1>{this.state.data}</h1>
                <button onClick={() => this.apple()}>Update Class
Component</button>
            </div>
        );
    }
}
```

5. Interview question.

Q :- Can, We used the useState() inside the class component.

Ans :- No, useState() not used inside the class component.

*/

```
/*
***** Props (Properties) with Functional Component *****
*/
```

1. What is Props.

Ans :- When pass the data inside the Component then we used the Props.
It's work like a paramenter.

2. Use of Props.

Ans :-

```
App.js
function App() {
    return (
        <div className="App">
            <h1>Hello App File</h1>
        </div>
    );
}
```

```

        <User />
        <User_class_compnt />
        <User_JSX />
        <Props_example name="Gurjeet" /> // name="Gurjeet" :-
name is a parameter
        Multiple parameter
        <Props_example name="Gurjeet" email="test@gamil.com" />
        Multiple parameter in single object
        <Props_example name="Gurjeet" email="test@gamil.com"
other={{address:"Chandigarh", mobile_no:111}} />
    </div>
    );
    }
    // Dynamically update data.
    import { useState } from 'react';
    function App() {
        const [name, setName] = useState("Gurjeet") ;
        return (
            <div className="App">
            <h1>Hello App File</h1>
            <Props_example name={name} />
            <button onClick={() => setName("Singh")}>Update
Props</button>
            </div>
            );
        }
    }
    Props_example.js
    export default function Props_example(props){ // Access with
parameter.
    //console.log(props);
    return(
        <div style={{backgroundColor:"green"}}>
        <h1>Props Component</h1>
        <h3>{props.name}</h3> { /* Get the value :- props.name
        <h4>{props.email}</h4>
        <h5>{props.other.address}</h5>
        <h6>{props.other.mobile_no}</h6>
        </div>
        )
    }

```

3. Example with Functional Component.

```

Ans :- export default function Props_example(props){ // Access with
parameter.
    //console.log(props);
    return(
        <div style={{backgroundColor:"green"}}>
        <h1>Props Component</h1>
        <h3>{props.name}</h3> { /* Get the value :- props.name
        <h4>{props.email}</h4>
        <h5>{props.other.address}</h5>
        <h6>{props.other.mobile_no}</h6>
        </div>
        )
    }

```

4. Update Props on button click.

```

Ans :- // Dynamically update data.
    import { useState } from 'react';
    function App() {

```

```

        const [name, setName] = useState("Gurjeet") ;
        return (
        <div className="App">
        <h1>Hello App File</h1>
        <Props_example name={name} />
        <button onClick={() => setName("Singh")}>Update
Props</button>

        </div>
        );
    }
    Props_example.js
    export default function Props_example(props){ // Access with
parameter.
    //console.log(props);
    return(
        <div style={{backgroundColor:"green"}}>
        <h1>Props Component</h1>
        <h3>{props.name}</h3> { /* Get the value :- props.name
        <h4>{props.email}</h4>
        <h5>{props.other.address}</h5>
        <h6>{props.other.mobile_no}</h6>
        </div>
        )
    }

```

5. Interview question.

Q :- Can, We pass the function as a props?

Ans :- Yes, This concept is listing UpState.

```

    function App() {
        const [name, setName] = useState("Gurjeet") ;
        // For exampe :-
        function apple(){
            alert("Props");
        }
        return (
        <div className="App">
        <h1>Hello App File</h1>
        <Props_example name={name} />
        <button onClick={() => setName("Singh")}>Update
Props</button>

        </div>
        );
    }

```

<!-- Pass function as Props -->

App.js file

import Pass_function_as_props from './Pass_function_as_props';

```

function App() {
    function getData(){
        alert("Function as a Props");
    }
    return (
        <div className="App">
            <h1>Function as Props</h1>
            <Pass_function_as_props data={getData} />
        </div>
    );
}

```

Pass_function_as_props.js File

```

import React from 'react';
export default function Pass_function_as_props(props){
  return(
    <div>
      <h1>Pass function as Props</h1>
      <button onClick={() => props.data()}>Call Data
function</button>
    </div>
  )
}

```

Q :- Two component and Can we call the function inside a nested component.

Ans :- Yes, Call a function as props anywhere.

*/

/* ***** Props With Class Component ***** */

1. What is Props.

Ans :- When passed the data from One component to another component. Props(parameter).

2. Use of Prpos.

Ans :- <Props_with_class_compnt name={this.state.name} email="test@gmail.com"></Props_with_class_compnt>

3. Example with Class Component.

Ans :- App.js file

```

import Props_with_class_compnt from 'Props_with_class_compnt'
class App extends Component{
  constructor(){
    super();
    this.state={
      name : "Amit"
    }
  }

  render(){
    return (
      <div className="App">
        <h1>Props With Class Component</h1>
        <Props_with_class_compnt name={this.state.name}
email="test@gmail.com"></Props_with_class_compnt>
        Multiple Values
        <Props_with_class_compnt name="Prabhjot"
email="prabhjot@gmail.com"></Props_with_class_compnt>
        <Props_with_class_compnt name="Harshdeep"
email="harshdeep@gmail.com"></Props_with_class_compnt>
        <button onClick={() => this.setState({name : "Sameer"})}>Class
Props Changed</button>
      </div>
    );
  }
}

```

Props_with_class_compnt.js file

```

import React, {Component} from 'react';
export default class Props_with_class_compnt extends Component{

```

// When we used class component Render() is must. Without
Render() class component not run.

```
render() {  
    // Check how many props are coming.  
    //console.log(this.props);  
    return (  
        <div>  
            Props With Class component  
            <h1>Name :- {this.props.name}</h1>  
            <h3>Email :- {this.props.email}</h3>  
        </div>  
    )  
}
```

4. Update props with button click.

5. Interview question.

Q :- Can, Change the value of props in the component where we received it?

Ans :- No, You will change the props where to you send the value.

Q :- Can, We pass the HTML in component in the App.js file. Like this

<Props_with_class_compnt><h1>Pass HTML </h1></><Props_with_class_compnt>.

If we pass. So how to get in the component file :-

Props_with_class_compnt.js

Ans :- Yes,

1. Using in the class component

this.props.children :- this we pass the HTML in the component.

2. Using in the functional component.

this.children :- this we pass the HTML in the component.

*/

/*
***** Get Input Box Value *****
*/

1. Make input field.

Ans :- <input type='text' onChange={getInputData}/>

2. Make function and get value.

```
Ans :- function App() {  
    // Print the value using State. Defined state  
    const [data, setData] = useState(null);  
    // Defined state on click  
    const [print, setPrint] = useState(false);  
    // Get value on change  
    function getInputData(val) {  
        //console.log(val.target.value);  
        setData(val.target.value);  
    }  
    return (  
        <div className="App">  
            <h1>Get input box value</h1>  
            //<User />  
            //<User_class_compnt />  
            //<User_JSX />  
            <Props_example />  
            <input type='text' onChange={getInputData}/>  
            /* Get the value from input box  
            <h3>Input value On change function :- {data}</h3>  
            //Change the value on button click
```

```

    <button onClick={() => setPrint(true)}>Input Value</button>
    <h3>Input value On click function :-
    {
      print?
      <h4>{data}</h4>
      :null
    }</h3>
  </div>
);
}
3. Make button and State.
Ans :- function App() {
  // Defined state on click
  const [print, setPrint] = useState(false);
  // Get value on change
  function getInputData(val){
    //console.log(val.target.value);
    setData(val.target.value);
    // To show and hide value
    setPrint(false);
  }
  return (
    <div className="App">
      <h1>Get input box value</h1>
      //<User />
      //<User_class_compnt />
      //<User_JSX />
      <Props_example 7>
      <input type='text' onChange={getInputData}/>
      //Change the value on button click
      <button onClick={() => setPrint(true)}>Input Value</button>
      <h3>Input value On click function :-
      {
        print?
        <h4>{data}</h4>
        :null
      }</h3>
    </div>
  );
}

```

4. Get value on button click.

```

Ans :- <h3>Input value On click function :-
      {
        print?
        <h4>{data}</h4>
        :null
      }</h3>

```

5. Interview question.

Q :- Can we get the checkbox or radion button value?

Ans :-

*/

```

/***** Hide and Show element *****/
/*

```

1. Make a div or any other element.

```

Ans :- {
  status ? <h1>Hello App File</h1> : null
}

```

2. Make state for hiding and showing element.

```
Ans :- // Defined State
        // const [status, setStatus] = useState(false);
        // Using single click button
        // const [status, setStatus] = useState(true);
```

3. Make button.

```
Ans :- // Using multiple button
        <button onClick={() => setStatus(false)}>Hide</button>
        <button onClick={() => setStatus(true)}>Show</button>
        // For Single button
        <button onClick={() => setStatus(!status)}>Hide and Show
        (Toggle)</button>
```

4. Update state on button click.

```
Ans :- function App() {
        // Defined State
        // const [status, setStatus] = useState(false);
        // Using single click button
        const [status, setStatus] = useState(true);
        return (
        <div className="App">
        // Using multiple button
        {
                status ? <h1>Hello App File</h1> : null
        }
        <button onClick={() => setStatus(false)}>Hide</button>
        <button onClick={() => setStatus(true)}>Show</button>

        // Using single button
        {
                status ? <h1>Hello App File</h1> : null
        }
        <button onClick={() => setStatus(!status)}>Hide and Show
        (Toggle)</button>
        </div>
        );
    }
```

5. Interview question.

Q :- Can, We used the same method in ReactJS or JQuery for hide and show element.

Ans :-

*/

```
/****** Basic Form *****/
/*
```

1. Make HTML form.

App.js file

```
Ans :- <form onSubmit={getFormData}>
        </form>
```

2. Add some field.

```
Ans :- <form onSubmit={getFormData}>
        <input type='text' placeholder='Enter Name' onChange={(e) =>
        setName(e.target.value) }/> <br />
        <select onChange={(e) => setInteraset(e.target.value)}>
        <option>Select Option</option>
        <option>Marvel</option>
        <option>DC</option>
        </select><br />
```

```

        <input type='checkbox' onChange={ (e) =>
setTnc(e.target.checked)} /><span>Terms and Condition..</span><br />
        <button type='submit'>Submit</button>
    </form>

```

3. Use state for field data.

Ans :- // Show the value on every fields. So we defined the State.

```

    const [name, setName] = useState("");
    const [tnc, setTnc] = useState(false);
    const [interiset, setInteriset] = useState("");

```

4. Submit and Control form.

Ans :- function getData(e){ // e mean's event
 e.preventDefault(); // This stop the form submission.
 console.log(name,tnc,interiset);
}

Full Example :-

```

function App() {
    // Show the value on every fields. So we defined the
State.

    const [name, setName] = useState("");
    const [tnc, setTnc] = useState(false);
    const [interiset, setInteriset] = useState("");

    function getData(e){ // e mean's event
    e.preventDefault(); // This stop the form submission.
    console.log(name,tnc,interiset);
    }
    return (
    <div className="App">
    <h1>Handle Form in ReactJs</h1>
    <form onSubmit={getData}>
        <input type='text' placeholder='Enter Name'
onChange={ (e) => setName(e.target.value) } /> <br />
        <select onChange={ (e) =>
setInteriset(e.target.value) }>
            <option>Select Option</option>
            <option>Marvel</option>
            <option>DC</option>
        </select><br />
        <input type='checkbox' onChange={ (e) =>
setTnc(e.target.checked)} /><span>Terms and Condition..</span><br />
        <button type='submit'>Submit</button>
    </form>
    // If you are not create a form. But you will be submit
the data
    //When we submit the form it's reload and it's handle
with call the event in this form onSubmit()
    </div>
    );
}

```

5. Interview question.

Q :- How to disabled the field.

*/

```

/***** Conditional Rendering | If condition *****/
/*

```

1. Make Component.

Ans :- Conditional_rendering.js file
 import { useState } from "react";

```

export default function Conditional_rendering(){
//const [loggedIn, setLoggedIn] = useState(true);
// Not Recommend method
// if(loggedIn){
//     return(
//         <div><h1>Welcome User Profile</h1></div>
//     )
// } else {
//     return(
//         <div><h1>Profile Component</h1></div>
//     )
// }

// Recommend Method
// return(
//     <div>
//         {loggedIn?<h1>Welcome User
Profile</h1>:<h1>Profile component</h1>}
//     </div>
// );

// If{}if else{}else{} Condition
//     const [loggedIn, setLoggedIn] = useState(1);
//     return(
//         <div>
//             {loggedIn == 1 ? <h1>Welcome Admin Profile</h1> :
loggedIn == 2 ? <h1>Welcome User Profile</h1> : <h1>Profile
component</h1>}
//         </div>
//     );
//     <h1>Profile Component with Conditional Rendering</h1>
// }

```

2. Use State.

Conditional_rendering.js file

```

Ans :- // import { useState } from "react";
//const [loggedIn, setLoggedIn] = useState(true);

```

App.js

```

Ans :- import Conditional_rendering from "../Conditional_rendering";
<Conditional_rendering />

```

3. Not Recommended way for conditional.

```

Ans :- //const [loggedIn, setLoggedIn] = useState(true);
// Not Recommend method
// if(loggedIn){
//     return(
//         <div><h1>Welcome User Profile</h1></div>
//     )
// } else {
//     return(
//         <div><h1>Profile Component</h1></div>
//     )
// }

```

4. Correct way Condition.

```

Ans :- // Recommend Method
// return(
//     <div>
//         {loggedIn?<h1>Welcome User
Profile</h1>:<h1>Profile component</h1>}
//     </div>

```

```

// );
5. Interview question.
Q :- If we are used the variable instead of useState the condition are
working or not?
Ans :-
*/

/***** Life cycle method *****/
/*
Q :- How many phase in Life cycle method?
Ans :- Three phase in Life cycle method. 1. Mounting 2. Updating 3.
Unmounting.
    Example :- When we load the application after that the popup is
call. Popup is showing using the life cycle method.
Q :- Where to use the life cycle component?
Ans :- All the life cycle method used inside the component. Like :-
Functional and Class component.
*/

/***** Life Cycle method in ReactJs *****/
/*
1. When component is loaded.
2. When component is updated with state and props.
3. When component is removed.
4. Interview question.
Q :- When we removed the component and reload again then my new life
cycle is run or the old life cycle is run?
Ans :- New life cycle is run.
*/

/***** Constructor Life Cycle Method *****/
/*
1. What is constructor.
Ans :- When we call the before the component. Then we used the
constructor.
    super() :- This is used for parent constructor like component
class for calling.
2. Why is this important.
3. What we can do in this life cycle method.
4. Interview question.
Q :- Is constructor is life cycle method or not?
Ans :- Yes, Definitely is life cycle method.
*/

/***** Render, Life Cycle Method *****/
/*
1. Use of Render method.
Ans :- Render is explain like executer and represented. All HTML is
written inside the render method.
    When the something is changed then render is again run.
    When the render method is call :- 1. When the component is ready.
                                     2. When the state update.
                                     3. When the props update.
2. Make class component.
Ans :- import React from "react";
       export default class Render_in_react extends React.Component{

}

```

3. Use state and props with Render.

Ans :- // Use a state

```
constructor() {
    super();
    this.state = {
        email : "test@gmail.com"
    }
}
render() {
    //console.log("Render Method call", this.props);
    console.log("Render Method call", this.state.email);
    return(
        <div>
            <h1>Render in React</h1>
            // <span>Render name :- {this.props.name}</span>
            <button onClick={() => this.setState({email :
"gurjeet@test.com"})}>Update Email</button>
        </div>
    )
}
```

4. Interview question.

Q :- Can, We call the props inside the constructor?

Ans :- Yes, console.log(this.props); used this for check the props.
*/

/*
/***** ComponentDidMount Life Cycle method (Mount means all
the HTML and CSS are ready) *****/

/*

1. Use of componentDidMount method.

Ans :- componentDidMount is used when we call the api. Suppose we load
the list in to the page. Any of operation like Hide and Show and based on
HTML all are done inside this.

2. Make class component.

Ans :-

```
class App extends React.Component {
    constructor() {
        super();
        //console.log("Constructor");
        this.state={
            name : "Gurjeet"
        }
    }
    componentDidMount() {
        console.log("Component Did Mount");
    }
    render() {
        console.log("Render");
        return(
            <div className='App'>
                <h1>Component Did Mount</h1>
                <span>{this.state.name}</span>
                <button onClick={() => {this.setState({name :
"Singh"}})}>Update name</button>
            </div>
        );
    }
}
```

```

    }
    export default App;

```

3. No Effect of state and props.

4. Interview question.

Q :- Why we used the component Did Mount?

Ans :- Call the API is best example of component Did Mount.

Q :- Render and Component Did Mount which one is first call?

Ans :- First Render call and after that ComponentDidMount is call.

Q :- Differnec between compoentDidMount and componentDidUpdate?

Ans :- ComponentDidMount is runing when our HTML is ready. And ComponentDidUpdate when are states report is updated.

Q :- Inside the Render. Can we update the state?

Ans :- No, Because Render is call every time and it's going to infinite loop.

*/

```

/***** componentDidUpdate, Life cycle method *****/
/*

```

1. Use of componentDidUpdate method.

Ans :- Do update directly. When you update the componentDidUpdate using the conditions.

2. Make class component.

App.js

Ans :- class App extends React.Component {

```

    constructor(){
        super();
        console.log("constructor");
        this.state = {
            //name : "Gurjeet"
            count : 0
        }
    }

```

```

    componentDidUpdate(prvProps, prvState, snapshot){
        // prvState :- Provide the pervious state.
        //console.log("Component Did Update.",
        prvState.count,this.state.count);

```

```

//        if(prvState.count===this.state.count){
//            alert("Data is already same");
//        }
//        // Same as performe using a prvProps
//        // Going to infinite loop
//        this.setState({
//            count : this.state.count+1
//        })
//        // To handle this situation with condition
//        if(this.state.count > 10){
//            this.setState({
//                count : this.state.count+1
//            })
//        }
//        // snapshot :- If we defined a use the getsnapShortBeforeUpdate()
//        function is used then we get the value. Otherwise it's showing the
//        undefined.
//        // getsnapShortBeforeUpdate() this method give the snapshot.
//        console.log(snapshot);
//    }

```

```

render() {
  console.log("render");
  return (
    <div className='App'>
      <h1>
        Component Did Update {this.state.count}
      </h1>
      <button onClick={() => this.setState({name : "Singh"})}>Component
Did Update</button>
      <button onClick={() => this.setState({count : this.state.count+1
})}>Component Did Update</button>
      <button onClick={() => this.setState({count : 1 })}>Component Did
Update</button>
      <button onClick={() => this.setState({count : 1 })}>Component Did
Update</button>
    </div>
  )
}

```

3. Use with State and Props.
4. Previous Props and State.

5. Interview question.

Q :- Can we update the component with componentDidMount?

Ans :- Yes, We can update the componentDidMount with condition. Without condition is going to infinite loop.

Q :- Can We stop the componentDidMount() for calling?

Ans :- Using a shouldComponentUpdate() and give the return false keyword. Yes, Definitely we stop the calling of componentDidMount().

Q :- In componentDidMount() can we call the API or not?

Ans :- Yes, But when you call a condition then. Otherwise you will not called.

*/

/****** shouldComponentUpdate (Life cycle method)

******/

/*

1. When shouldComponentUpdate called and use.

Ans :- This is stand in the updating state. shouldComponentUpdate component put the question. Like can we update the component or not. And we also put the condition when we update the component or not.

Benifits :-

1. You can stop the component in spacific condition.

```

    if(this.state.count > 5 && this.state.count < 10){
      return true;
    }

```

2. It can stop Rendering.

App.js

Ans :- class App extends React.Component {

```

  constructor(){
    super();
    this.state = {
      count : 0
    }
  }

```

```

  shouldComponentUpdate(){
    console.log("Should Component Update", this.state.count);
  }
}

```

```

    // By default it return false. It's not update the component.
    return true;
  }

  render() {
    return (
      <div className='App'>
        <h1>Should Component Update {this.state.count}</h1>
        <button onClick={() => this.setState({count :
this.state.count +1})}>Should Component Update</button>
      </div>
    );
  }
}

```

3. Use with state and props.

4. Interview question.

Q :- Which one is first call shouldComponentUpdate() or
componentDidUpdate()?

Ans :- shouldComponentUpdate() called first if the inside the return
true.

*/

/* ***** componentWillUnmount() (Life Cycle method) */
/*

1. When componentWillUnmount() called.

Ans :- When our component is removed from DOM then componentWillUnmount()
is called.

2. Example of Component Will Unmount.

Ans :- App.js

```

import StudentComponentWillMount from
'./StudentComponentWillMount';
class App extends React.Component {
  constructor() {
    super();
    this.state = {
      show : true
    }
  }
  render() {
    return (
      <div className='App'>
        {
          this.state.show ? <StudentComponentWillMount /> :
<h1>Child Component Remove</h1>
        }
        <button onClick={() =>
this.setState({show:!this.state.show})}>Toggle Child Component</button>
      </div>
    );
  }
}
StudentComponentWillMount.js
import React from "react";
class StudentComponentWillMount extends React.Component {
  componentWillUnmount() {
    alert("Component Will Unmount Called");
  }
  render() {

```

```

        return(
        <div className='App'>
            <h1>Student Component Will Unmount</h1>
        </div>
        );
    }
}
export default StudentComponentWillMount;

```

3. Use of Component Will Unmount.

Ans :- If you are used API and suddently you will remove the component then used this. Cancel Data loading, Called a function, If you are used setTime() function

and you want removed this. Everything like Event, function you will called here.

4. Interview Question.

Q :- componentWillUnmount() called just before or after when the component is removed.

Ans :- just before called.

*/

```

/***** Hooks in ReactJs *****/
/*

```

1. What are the Hooks in Reactjs.

Ans :- With Hooks, We can use class component feature in functional component such as state, life cycle, pure component, etc.

import React,{useState} from 'react'; // useState() is hook. When you type the use keyword then you will see the hook list. When you create a custom hooks the write the use keyword before the hook name this is a reserved keyword.

2. How to use them.

Ans :- const [data, setData] = useState("Singh");

3. Example of React Hooks with useState.

```

Ans :- function App(){
    const [data, setData] = useState("Singh"); // This is hook
defined different different hook are defined differently.
    return(
    <div className='App'>
    <h1>{data}</h1>
    <button onClick={ () => setData("Gurjeet")}>Update Data</button>
    </div>
    );
}

```

4. Interview question.

Q :- Can we called the hooks inside the class component?

Ans :- No, We don't called inside the class component.

*/

```

/***** useEffect in ReactJs *****/
/*

```

1. What is useEffect.

Ans :- useEffect() is life cycle method in functional component. When are component is created, state update and Props then it's called. You will used the useEffect() with multiple time.

2. How to use useEffect.

Ans :-

```

// Declaration with different way
React.useEffect( () => {

```

```

    })
    import React,{useEffect} from 'react';
    useEffect( () => {

    })

```

3. Example useEffect.

App.js

```

Ans :- function App(){
    const [count, setCount] = useState(0);
    useEffect( () => {
        console.log("Use Effect");
    }) // [] square barcket is used for condition inside the
useEffect() function.
    return (
        <div className='App'>
            <h1>useEffect in ReactJs {count}</h1>
            <button onClick={ () => setCount(count + 1) }>Update
Counter</button>
        </div>
    );
}

```

4. useEffect with State and Props.

5. Interview question.

*/

```

/***** useEffect with Spacific State and Props *****/
/*

```

1. What is useEffect.

2. useEffect with spacific State.

Ans :- App.js

```

    function App(){
        const [data, setData] = useState(10);
        const [count, setCount] = useState(100);
        //      useEffect(() =>{
        //          console.log("Use Effect Called");
        //      });
        // To used the useEffect hook in a particular state with
[state_name]
        useEffect(() => {
            console.log("Called Data with useEffect");
        },[data]);
        useEffect(() => {
            alert("Count :-"+ count);
        },[count]);
        // To used the useEffect hook in a particular Props with
[props.data]
        return (
            <div className='App'>
                <h1>State Count {count}</h1>
                <h1>State data {data}</h1>
                <button onClick={ () => setCount(count + 1) }>Update
Count</button>
                <button onClick={ () => setData(data + 1) }>Update
data</button>
            </div>
        )
    }

```

```
export default App;
3. useEffect with specific Props.
```

Ans :- App.js

```
import { useState, useEffect } from 'react';
import UseEffect_with_props from './UseEffect_with_props';
function App(){
  const [count, setCount] = useState(100);
  const [data, setData] = useState(10);
  return (
    <div className='App'>
      <UseEffect_with_props count={count} data={data} />    //
```

Props is defined

```
      <button onClick={() => setCount(count + 1)}>Update
```

Count</button>

```
      <button onClick={() => setData(data + 1)}>Update
```

data</button>

```
    </div>
  )
}
```

UseEffect_with_props.js

```
import { useState, useEffect } from 'react';
function UseEffect_with_props(props){
  useEffect(() =>{
    alert("Count is ==" + props.count);
  }, [props.count]);
  return (
    <div className='App'>
      <h1>State Count Prpos {props.count}</h1>
      <h1>State data Prpos {props.data}</h1>

    </div>
  )
}
export default UseEffect_with_props;
```

4. Interview question.

Q :- When we pass the props using Redux then the useEffect working like it's normally work.

Ans :-

*/

```
/* ***** Style Type in ReactJS ***** */
/*
```

1. How many way we have for style in ReactJs.

2. Use the normal Stylesheet.

Ans :- App.js

```
import './style.css';
<h1 className='primary'>Style type 1 React JS</h1>
```

3. Style with the variable or Inline style.

Ans :- App.js

```
  <h1 className='' style={{color:'red',
backgroundColor:"black"}}>Style type 2 React JS</h1> // {} :- First curle
barces is JSX property. {} second braces is object. In the CSS we used
the property like "background-color" But in the ReactJs we used the camel
case for property like "backgroundColor"
```

4. CSS with Module.

Ans :- App.js

```
import style from './custom.module.css';
<h1 className={style.success}>Style type 3 React JS</h1> //
```

Style with Module file.module.css. module is important to used with the CSS file name when we used the module. custom.module.css

```
custom.module.css
.success{
  color:gold;
  background-color:black
}
```

5. Interview question.

Q :- Can, We used the together normal CSS and modular CSS?

Ans :- Yes.

*/

/***** Install Bootstrap *****/

1. What is Bootstrap.

Ans :- Bootstrap is a CSS libaray.

2. Install Bootstrap.

Ans :- npm install react-bootstrap bootstrap.

3. Use and register Bootstrap libaray.

Ans :- // Bootstrap CSS index.js

```
import 'bootstrap/dist/css/bootstrap.min.css';
// Import Bootstrap Component
import Button from 'react-bootstrap/Button';
```

4. Example with Bootstrap component.

Ans :-

```
// Import Bootstrap Component
import Button from 'react-bootstrap/Button';
<div className='App'>
  <h1>Install Bootstrap</h1>
  <Button variant='primary' onClick={() => alert("Primary
Button")}>Primary Button</Button>
</div>
```

5. Interview question.

Q :- Can we have libaray like a bootstrap?

Ans :- Yes, Tailwind CSS etc.

*/

/***** Handle Array with List *****/

1. Make an array.

2. Why use map() function, not for loop.

Ans :- When use the return then map() function is support but for loop not supported.

```
const students = ["Gurjeet", "Harshdeep", "Prabhjot", "Abhi"];
// // map looping
// students.map((item) => {
//   console.log("My name is = "+item);
// });
// // For loop
// for(let i = 0; i<students.length; i++){
//   console.log("My name is for loop = "+ students[i]);
// }
```

3. Make list with an array.

```
Ans :- <div className='App'>
      <h1>Handle Array with List</h1>
      {
        // Using the map() function.
        students.map((item) =>
          //console.log(item);
          <h6>My name is {item}</h6>
        )
      }
    </div>
```

4. Make list with an array of object.

```
Ans :- function App(){
  // Array Objects
  const students = [
    {
      name : "Gurjeet",
      email : "test@demo.com",
      contact : 321564789
    },
    {
      name : "Harshdeep",
      email : "harshdeep@demo.com",
      contact : 999999999
    },
    {
      name : "Prabhjot",
      email : "prabhjot@demo.com",
      contact : 888888888
    },
  ]
  return (
    <div className='App'>
      <h1>Handle Array with List</h1>
      <table border="1">
        <thead>
          <tr>
            <td>Name</td>
            <td>Email</td>
            <td>Contact</td>
          </tr>
        </thead>
        {
          // Using the map() function.
          students.map((item) =>
            //console.log(item);
            <tr>
              <td>{item.name}</td>
              <td>{item.email}</td>
              <td>{item.contact}</td>
            </tr>
          )
        }
      </table>
    </div>
  )
}
```

5. Interview question.

Q :- Warning: Each child in a list should have a unique "key" props?

Ans :-

*/

```
/*
/***** List with Bootstrap and Unique Key *****/
/*
```

1. Make list form Array.

Ans :- // Array Objects

```
const users = [
  {name : "Gurjeet", email : "test@demo.com", contact : 321564789},
  {name : "Harshdeep", email : "harshdeep@demo.com", contact :
999999999},
  {name : "Prabhjot", email : "prabhjot@demo.com", contact :
888888888},
]
```

2. What is Unique key in loop.

Ans :-

```
// Without Condition
users.map((item, i) =>
  <tr key={i}> // Using key={i} Remove this Warning: Each
child in a list should have a unique "key" props
    <td>{item.name}</td>
    <td>{item.email}</td>
    <td>{item.contact}</td>
  </tr>
)

}

// With condition
{
  users.map((item, i) =>
    item.contact === 888888888 ?
    <tr key={i}> // Using key={i} Remove this Warning: Each
child in a list should have a unique "key" props
      <td>{item.name}</td>
      <td>{item.email}</td>
      <td>{item.contact}</td>
    </tr>
    :null
  )
}
```

3. Install Bootstrap.

4. Use Bootstrap table with loop

Ans :- function App(){

```
// Array Objects
const users = [
  {name : "Gurjeet", email : "test@demo.com", contact : 321564789},
  {name : "Harshdeep", email : "harshdeep@demo.com", contact :
999999999},
  {name : "Prabhjot", email : "prabhjot@demo.com", contact :
888888888},
]
var sno = 1;
return (
  <div className='App'>
    <h1>List with Bootstrap and Unique Key</h1>
    <Table striped hover variant='dark'>
```

```

        <thead>
            <tr>
                <td>S no.</td>
                <td>Name</td>
                <td>Email</td>
                <td>Contact</td>
            </tr>
        </thead>
        <tbody>
            // Without Condition
            {
                users.map((item, i) =>
                    <tr key={i}> // Using key={i} Remove this Warning: Each
child in a list should have a unique "key" props
                        <td>{sno++}</td>
                        <td>{item.name}</td>
                        <td>{item.email}</td>
                        <td>{item.contact}</td>
                    </tr>
                )
            }
        </tbody>
    </Table>
</div>
)
}

```

5. Interview Question.

Q :- Show the serial number?

Ans :-

*/

/****** Nested List with Nested Array *****/

/*

1. Make Nested Array.

Ans :- const users = [

```

    {
        name: "Gurjeet", email: "test@demo.com", address: [
            { hn: 10, city: "Nodia", country: "India" },
            { hn: 34, city: "Chandigarh", country: "India" },
            { hn: 43, city: "Delhi", country: "India" },
        ]
    },
    {
        name: "Harshdeep", email: "harshdeep@demo.com", address: [
            { hn: 10, city: "Nodia", country: "India" },
            { hn: 34, city: "Chandigarh", country: "India" },
            { hn: 43, city: "Delhi", country: "India" },
        ]
    },
    {
        name: "Prabhjot", email: "prabhjot@demo.com", address: [
            { hn: 10, city: "Nodia", country: "India" },
            { hn: 34, city: "Chandigarh", country: "India" },
            { hn: 43, city: "Delhi", country: "India" },
        ]
    },
]

```

2. Understand Nested Array.

Ans :-

3. Use the map() function.

```
Ans :- users.map((item, i) =>
    <tr key={i}> // Using key={i} Remove this Warning: Each
child in a list should have a unique "key" props
    <td>{sno++}</td>
    <td>{item.name}</td>
    <td>{item.email}</td>
    <td>{item.contact}</td>
  </tr>
)
```

4. Use nested Map function.

```
Ans :- {
  users.map((item, i) =>
    <tr key={i}>
      <td>{i+1}</td>
      <td>{item.name}</td>
      <td>{item.email}</td>
      <td><Table striped hover variant='dark'>
        <thead>
          <tr>
            <td>House No.</td>
            <td>City</td>
            <td>Country</td>
          </tr>
        </thead>
        <tbody>
          {
            item.address.map((data) =>
              <tr>
                <td>{data.hn}</td>
                <td>{data.city}</td>
                <td>{data.country}</td>
              </tr>
            )
          } </tbody>
        </Table>
      </td>
    </tr>
  )
}

// App.js
function App() {
  // Array Objects
  const users = [
    {
      name: "Gurjeet", email: "test@demo.com", address:
[
    { hn: 10, city: "Nodia", country: "India" },
    { hn: 34, city: "Chandigarh", country: "India" },
    { hn: 43, city: "Delhi", country: "India" },
  ]
    },
    {
      name: "Harshdeep", email: "harshdeep@demo.com",
address: [
```

```

        { hn: 10, city: "Nodia", country: "India" },
        { hn: 34, city: "Chandigarh", country: "India" },
        { hn: 43, city: "Delhi", country: "India" },
      ],
    },
    {
      name: "Prabhjot", email: "prabhjot@demo.com",
      address: [
        { hn: 10, city: "Nodia", country: "India" },
        { hn: 34, city: "Chandigarh", country: "India" },
        { hn: 43, city: "Delhi", country: "India" },
      ],
    },
  ],
  return (
    <div className='App'>
      <h1>Nested List with Nested Array</h1>
      <Table striped hover variant='dark'>
        <thead>
          <tr>
            <td>S No.</td>
            <td>Name</td>
            <td>Email</td>
            <td>Address</td>
          </tr>
        </thead>
        <tbody>
          {
            users.map((item, i) =>
              <tr key={i}>
                <td>{i+1}</td>
                <td>{item.name}</td>
                <td>{item.email}</td>
                <td><Table striped hover
variant='dark'>
                  <thead>
                    <tr>
                      <td>House No.</td>
                      <td>City</td>
                      <td>Country</td>
                    </tr>
                  </thead>
                  <tbody>
                    {
                      item.address.map((data)
=>
                        <tr>
                          <td>{data.hn}</td>
                          <td>{data.city}</td>
                          <td>{data.country}</td>
                        </tr>
                      )
                    }
                  </tbody>
                </Table>
              </td>
            </tr>
          )
        }
      </tbody>
    </Table>
  )
)

```

```

                </tbody>
            </Table>
        </div>
    )
}

```

5. Interview question.

*/

```

/***** Reuse Component *****/
/*

```

1. Make a component.

Ans :- Reuse_component.js

```

export default function Reuse_component(props){
    return(
        <div>
            <span className="reuse_component">User name :-
{props.data.name}</span>
            <span className="reuse_component">Email :-
{props.data.email}</span>
            <span className="reuse_component">Address :-
{props.data.address}</span>
            // <button onclick={ alert(props.data.name)}>Click
me</button>
        </div>
    )
}

```

2. Make a List from the Array.

```

Ans :- <h1>Reuse Component</h1>
    {
        users.map((item, i) =>
            <Reuse_component data={item}/>
        )
    }

```

3. Use a Component inside the Map() function.

```

Ans :- App.js
function App() {
    // Array Objects
    const users = [
        {name: "Gurjeet", email: "test@demo.com", address:111},
        {name: "Harshdeep", email: "harshdeep@demo.com", address:222},
        {name: "Prabhjot", email: "prabhjot@demo.com", address:333},
    ]
    return (
        <div className='App'>
            <h1>Reuse Component</h1>
            {
                users.map((item, i) =>
                    <Reuse_component data={item}/>
                )
            }
        </div>
    )
}

```

```

Reuse_component.js
export default function Reuse_component(props){
    return(
        <div>

```

```

        <span className="reuse_component">User name :-
{props.data.name}</span>
        <span className="reuse_component">Email :-
{props.data.email}</span>
        <span className="reuse_component">Address :-
{props.data.address}</span>
        //<button onclick={ alert(props.data.name)}>Click
me</button> /
        </div>
    )
}

```

4. Interview question.

Q :- Which file we defined the button when we used onclick method (App.js OR Reuse_component.js)?

Ans :- In App.js file we used like that function. Because how many time you called the component it's recalled this is impact the performance of application.

*/

/*
/***** React Fragment *****/
/*

1. What is React Fragment.

Ans :- This is pattern that was handle the multiple elements or multiple components. This is use as a wrapper.

The problem with this :- When we use the multiple elements without any wrapper like <div> it's showing error. But this is an extra <div> are available.

But we don't use this or hide this.

This problem is overcome by React Fragment.

2. How to use it.

Ans :- 3 ways to use it.

1. <React.Fragment>Do any thing</React.Fragment>
2. import React, { Fragment } from 'react'; Import React Fragment and use it <Fragment> </Fragment>
3. Use an empty or Blank tag <> </>

3. Use it with child component.

Ans :- App.js

```

import Fragment_cols from './Fragment_cols';
function App() {
  return (
    <div className='App'>
      <h1>React Fragment</h1>
      <Table>
        <tbody>
          <tr>
            <Fragment_cols />
          </tr>
        </tbody>
      </Table>
    </div>
  )
}

```

Fragment_cols.js

```

export default function Fragment_cols(){
  return (
    <>

```

```

        <td>Gurjeet</td>
        <td>Singh</td>
    </>
    )
}
4. Interview question.
Q :- Can, We used the className with fragment?
Ans :- No can't used the className. But we used the key inside the
fragment.
Q :- Can, we used the fragment inside the parents?
Ans :- Yes
*/

```

```

/***** Send Data Child to Parent Component ( Inside the Reactjs
that's call a (Lifting State Up)) *****/
/*

```

1. Make a Child Component.
2. How to pass Props parent to Child component.

```

Ans :- App.js // Parent data
import Lifting_state_up from './Lifting_state_up';
function App() {
  let data = "Gurjeet Singh";
  return (
    <div className='App'>
      <h1>Lifting State Up</h1>
      <Lifting_state_up name={data} />
    </div>
  )
}

```

```

Lifting_state_up.js // Get Data from Parent
export default function Lifting_state_up(props){
  return(
    <>
      <h2>User name :- {props.name}</h2>
      <button className="primary">Click me</button>
    </>
  );
}

```

3. Send the data child to parent.

4. Interview question.

Q :- Can, We used the App.js function inside the child or supre child component?

```

Example :- App.js
function parentAlert(data){
  //alert(data.name, data.email, data.contact);
  console.log(data);
}

```

Ans :- Yes, Like we pass the child to parent. Other option is Redux or Context API.

```

*/

/***** Pure Component *****/
/*

```

1. What is Pure Component.

Ans :- Pure component is a feature of ReactJs. We just import it form ReactJs.

Pure component is stop the Rerendering of component. This is check the first value and new value are same or not if the value is same then it not Rerendering the component.

Problem :-

```
class App extends React.Component {
  constructor() {
    super();
    this.state = {
      count : 1
    }
  }

  render() {
    console.log("Check Rerendering"); // Rerendering is running. This
    is problem.
    return (
      <div className='App'>
        <h1>Pure Component {this.state.count}</h1>
        <Button className="primary" onClick={() =>
this.setState({count:1})}>Click me</Button>
      </div>
    )
  }
}
```

```
// Resolve the problem with Pure Component.
import { PureComponent } from 'react';
// Using a State
class App extends PureComponent {
  constructor() {
    super();
    this.state = {
      count : 1
    }
  }

  render() {
    console.log("Check Rerendering"); // Rerendering is running. This
    is problem.
    return (
      <div className='App'>
        <h1>Pure Component {this.state.count}</h1>
        <Button className="primary" onClick={() =>
this.setState({count:1})}>Click me</Button>
        //Check the Pure Component is working
        <Button className="primary" onClick={() =>
this.setState({count:this.state.count + 1 })}>Click me</Button>
      </div>
    )
  }
}
```

2. How to use the Pure Component.

Ans :-

```
import { PureComponent } from 'react';
class App extends PureComponent
```

3. Pure component example with State.

Ans :- App.js

```

    // Using a State
    class App extends PureComponent {
    constructor() {
    super();
    this.state = {
        count : 1
    }
    }

    render() {
    console.log("Check Rerendering"); // Rerendering is running. This
is problem.
    return (
        <div className='App'>
        <h1>Pure Component {this.state.count}</h1>
        <Button className="primary" onClick={() =>
this.setState({count:1})}>Click me</Button>
        //Check the Pure Component is working
        <Button className="primary" onClick={() =>
this.setState({count:this.state.count + 1 })}>Click me</Button>
        </div>
    )
    }
    }

```

4. Pure component example with Props.

Ans :- App.js

```

import { PureComponent } from 'react';
// Using Props
import Pure_component_props from './Pure_component_props';
class App extends PureComponent {
    constructor() {
    super();
    this.state = {
        count : 1
    }
    }

    render() {
    return (
        <div className='App'>
        <Pure_component_props count={this.state.count}/>
        <Button className="primary" onClick={() =>
this.setState({count:this.state.count + 1 })}>Click me</Button>
        </div>
    )
    }
    }

```

```

Pure_component_props.js
import React,{ PureComponent } from 'react';
// Using Props
export default class Pure_component_props extends PureComponent {
    render() {
    console.log("Check User Rerendering"); // Rerendering is running.
This is problem.
    return (
        <div className='App'>
        <h1>Pure Component Props {this.props.count}</h1>

```

</div>

```
)  
}  
}
```

5. Interview question.

Q :- Can, We used the Pure Component inside the function component?

Ans :- useMemo Hook for used the Pure component insidet the functional component.

*/

```
/*  
***** useMemo Hook in ReactJs *****  
*/
```

1. Why we use useMemo Hook.

Ans :- useMemo Hook inhance the Application performance. Like class based component. We used the Pure component.

Same way we used useMemo Hook in functional component.

When we work with State and Porps then our component is unwantedly updating

2. Understand the problem.

Ans :- App.js

```
function App(){  
  const [count, setCount] = useState(0);  
  const [item, setItem] = useState(10);  
  function multiCount(){  
    return count * 5;  
  }  
  // The proble is when we click Item button multiCount() function  
called.  
  return (  
    <div className='App'>  
      <h1>useMemo Hook in ReactJs</h1>  
      <h2>Count :- {count}</h2>  
  
      <Button onClick={() => setCount(count + 1)}>Update  
Count</Button><br />  
      <h2>Item :- {item}</h2>  
      <Button onClick={() => setItem(item * 10)}>Update  
Item</Button>  
  
      <h2>{multiCount()}</h2>  
    </div>  
  );  
}
```

Solution :-

```
import { useMemo } from 'react';  
function App(){  
  const [count, setCount] = useState(0);  
  const [item, setItem] = useState(10);  
  const multiCountMemo = useMemo( function multiCount(){  
    console.log("Use memo Hook");  
    return count * 5;  
  },[count]) // [] Set the condition when the function is run.  
  // The proble is when we click Item button multiCount() function  
called.  
  return (  
    <div className='App'>  
      <h1>useMemo Hook in ReactJs</h1>
```

```

        <h2>Count :- {count}</h2>
        <h2>{multiCountMemo}</h2>

        <Button onClick={() => setCount(count + 1)}>Update
Count</Button><br />
        <h2>Item :- {item}</h2>
        <Button onClick={() => setItem(item * 10)}>Update
Item</Button>
    </div>
  );
}

```

3. Apply useMemo.

```

Ans :- import { useMemo } from 'react';
      const multiCountMemo = useMemo( function multiCount(){
        console.log("Use memo Hook");
        return count * 5;
      },[count]) // [] Set the condition when the function is run.

```

4. Interview question.

Q :- Can, We used the useMemo hook inside the class component?

Ans :-

*/

```

/***** Ref in ReactJs *****/
/*

```

1. Why use Ref.

Ans :- ReactJs asked use the Ref little as possible. Use this when the most important to used it. Other wise ignore it.

Ref directly manipulate the DOM. This is only used for functional component.

2. Make class component.

```

Ans :- class App extends React.Component{ }

```

3. Ref use with example.

```

Ans :- import { createRef } from 'react';
      class App extends React.Component{
        constructor(){
          super();
          this.inputRef = createRef()
        }
        componentDidMount(){
          //console.log(this.inputRef);
          // To manipulate the DOM
          //console.log(this.inputRef.current.value=1000);
          // To get the value form input text
        }
        getVal(){
          //console.log(this.inputRef);
          // To get the value form input text
          console.log(this.inputRef.current.value);
          // Change color
          this.inputRef.current.style.color="red";
          // Background color
          this.inputRef.current.style.backgroundColor = "green";
        }
        render(){
          return (
            <div className='App'>
              <h1>Ref in ReactJs</h1>
              <input type='text' ref={this.inputRef}/>
            </div>
          )
        }
      }

```

```

        // onfocuse, Change color etc
        <Button onClick={() => this.getVal()}>Ref Value</Button>
      </div>
    )
  }
}

```

4. Interview question.

Q :- Can, We used the multiple input Ref inside the component?

Ans :-

Q :- In the single component use the multiple Ref?

Ans :-

Q :- Use Ref inside the functional component?

Ans :-

*/

```

/***** useRef in ReactJs *****/
/*

```

1. Why use Ref.

Ans :- Hooks are only build for functional component.

2. Ref use with example.

```

Ans :- import { useRef } from 'react';
      function App(){
        const inputRef=useRef(null);
        function handleInputRef(){
          //alert("function call");
          inputRef.current.value=1000;
          inputRef.current.focus();
          inputRef.current.style.color="Red";
          inputRef.current.style.display="none";
        }
        return (
          <div className='App'>
            <h1>useRef in ReactJs</h1>
            <input type='text' ref={inputRef}/>
            <Button onClick={() => handleInputRef()}>Use Ref</Button>
          </div>
        )
      }

```

3. Focus, color, value handle with Ref.

```

Ans :- function handleInputRef(){
          //alert("function call");
          inputRef.current.value=1000;
          inputRef.current.focus();
          inputRef.current.style.color="Red";
          inputRef.current.style.display="none";
        }

```

4. Interview question.

Q :- How to handle the DOM?

Ans :- Using Ref we handle the DOM.

Q :- Can, We use the useRef or useState. And ReactJs is What recommend useRef or State?

Ans :- To use the useRef minmum to minmum. Because they directly manipulate the DOM then our application performance is slow. Always use in emergancy.

*/

```

/***** forwardRef in ReactJS *****/

```

```
/*
```

1. Why use forwardRef.

Ans :- forwardRef is a useRef advance version in ReactJs. When the input box and Button both are define in different component and access the DOM. Then we used the forwardRef. One component to second component. that's why we use forwardRef.

2. Make child component.

Ans :- ForwardRefExample.js

```
import React,{forwardRef} from "react"; // forwardRef using as a
wrapper like that forwardRef(function_name) and the value is get using
the "ref" parameter
```

```
function ForwardRefExample(props, inputRef){
  return (
    <div>
      <input type="text" ref={inputRef} />
    </div>
  )
}
```

```
export default forwardRef(ForwardRefExample);
```

3. Pass Ref with props and use it.

Ans :- App.js

```
import { useRef } from 'react';
import ForwardRefExample from './ForwardRefExample';
function App(){
  const inputRef = useRef(null);
  function updateInput(){
    inputRef.current.value = "1000";
    inputRef.current.style.color = "red";
    inputRef.current.focus();
    inputRef.current.style.backgroundColor = "green";
  }
  return (
    <div className='App'>
      <h1>forwardRef in ReactJS</h1>
      <ForwardRefExample ref={inputRef} />
      <Button onClick={updateInput}>Update input box</Button>
    </div>
  )
}
export default App;
```

4. Interview question.

Q :- To use the class component instead of the function component. What will be changes inside the Functional component file "ForwardRefExample.js"?

Ans :- Similar way to use class inside the forwardRef.

```
*/
```

```
/****** Controlled Component *****/
```

```
/*
```

1. What is Controlled component.

Ans :- In ReactJs when we controlled the input field using the state that's called controlled component.

And Uncontrolled component is handle directly with the DOM. Like jquery.

2. Make component.

Ans :- import { useState } from 'react';

3. Example of Controlled Component.

```
Ans :- function App(){
  const [val, setVal] = useState(0);
  return (
    <div className='App'>
      <h1>Controlled Component</h1>
      // This field is handle with the controlled or uncontrolled
method. Handle mean's there is a value and use it inside the API or send
any where. Then we handle this with state or controlled component//

      // <input type='text' value={val}/> // We can't change like this
value inside the browser. Then we do using the onChange function. //
      // <input type='text' value={val} onChange={(event) =>
setVal(event.target.value)}/> // This is controlled by the controlled
component. //
      // You can handle the default value like value={val} with
defaultValue="000" inside the input tag or set in the useState() function
like that useState(0). But you remove the value={val} from input tag
      <h3>Value = {val}</h3>
    </div>
  )
}
```

4. Interview question.

For knowlage :-

Q :- If someone is asked what is controlled or uncontrolled component?

Ans :- Then understand in this component like single or multiple input field.

Q :- Used the controlled component inside the class component?

Ans :- Controlled component used with both Functional and Class component.

*/

/***** Uncontrolled Component *****/

/*

1. What is uncontrolled Component.

Ans :- Uncontrolled component are involved some input fields and that input field are not controlled by ReactJs state. That's controlled with Javascript or Ref.

2. Make component.

3. Example of uncontrolled component.

Ans :- App.js

```
function App() {
  let nameRef = useRef(null);
  let secondnameRef = useRef(null);
  function submitForm(e) {
    e.preventDefault();
    console.log(`First name :- ${nameRef.current.value} Last name :-
${secondnameRef.current.value}`);
    //Uncontrolled without Ref
    let sirName = document.getElementById("sirName").value;
    console.log(`Sir name ${sirName}`);
  }
  return (
    <div className='App'>
      <h1>Uncontrolled Component</h1>
```

```

        <form onSubmit={handleSubmit}>
        <input type='text' ref={nameRef} /> <br /><br />
        <input type='text' ref={secondnameRef}/> <br /><br />
        <input type='text' id='sirName' /> <br /><br /> //
Uncontrolled without Ref
        <button>Submit</button>
        </form>
        </div>

    )
}

```

4. Interview question.

Q :- Some field are controlled by Uncontrolled Component and some fields controlled with State. So what would be say this Controlled Component or Uncontrolled Component.

Ans :- This is called Uncontrolled Component. Because this is little bit uncontrolled that's way this is Uncontrolled Component.

*/

/****** Simple HOC *****/

/*

1. What is HOC (High Order Component).

Ans :- HOC mean's First component use the second component as a props and return the differently component. HOC is a get the component as a input and output as a component.

2. Make 2 Component.

Ans :- You can do anything is here.

```

// HOC (High Order Component)
function HocRed(props){
  return <h2 style={{backgroundColor:"red", width:200}}><props.cmp
/></h2>
}
// Simple Component
function Counter(){
  const [count, setCount] = useState(0);
  return <div>
    <h4>Counter :- {count}</h4>
    <Button onClick={() => setCount(count + 1)}>Update
Counter</Button>
    </div>
  }
}

```

3. Example of HOC.

Ans :-

```

function App() {
  return (
    <div className='App'>
      <h1>Simple HOC</h1>
      // <Counter />
      <HocRed cmp={Counter}/> // cmp :- mean's component
      <HocGreen cmpGreen={Counter} />
      <HocBlue cmp={Counter} />
    </div>
  )
}

```

// If you used the conditions with single component and declared with multiple times. But how many conditions are we used. This situation handle with HOC. You can do anything here.

```
// HOC (High Order Component)
function HocRed(props){
  return <h2 style={{backgroundColor:"red", width:200}}><props.cmp
/></h2>
}
function HocGreen(props){
  return <h2 style={{backgroundColor:"green",
width:200}}><props.cmpGreen /></h2>
}
function HocBlue(props){
  return <h2 style={{backgroundColor:"blue", width:200}}><props.cmp
/></h2>
}
// Simple Component
function Counter(){
  const [count, setCount] = useState(0);
  return <div>
    <h4>Counter :- {count}</h4>
    <Button onClick={() => setCount(count + 1)}>Update Counter</Button>
  </div>
}
4. Interview question.
Q :- Where to used the HOC?
Ans :-
*/
```

```
/****** Routing Setup *****/
/*
```

1. What is Routing.

Ans :- Convert the component into pages and open in spacific link called Rounting.

2. Make 2 component.

```
Ans :- function Home(){
  return (
    <div>
      <h1>Home</h1>
      <p>This is a Home page.</p>
    </div>
  )
}

function About(){
  return (
    <div>
      <h1>About</h1>
      <p>This is a About page</p>
    </div>
  )
}
```

3. Install Routing Package.

Ans :- npm install react-route-dom.

```
import { BrowserRouter as Router, Routes, Route, Link } from
'react-router-dom';
```

4. Make Page.

```
Ans :- function App() {

  return (
    <div className='App'>
```

```

<h1>Routing Setup</h1>
<Router>
  <Link to="/home">Home</Link><br />
  <Link to="/about">About</Link>
  <Routes>
    <Route path='/home' element={<Home />} />
    <Route path='/about' element={<About />} />
    // This is generate the error
    // <Route path='/home'><Home /></Route>
    // <Route path='/about'><About /></Route>
  </Routes>
</Router>
</div>
)
}

```

5. Interview question.

Q :- If I put the div inside the APP function. It's working or not?

Ans :- Yes we can put the direct HTML inside the routes.

*/

/* ***** Routing Example with Best Practices ***** */

/*

1. Use React Router wrapper in index.js.

```

Ans :- Index.js
// Router
import { BrowserRouter as Router } from 'react-router-dom';
const root =
ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <Router>
      <App />
    </Router>
    //<Users />
  </React.StrictMode>
);

```

2. Navbar file for link.

```

Ans :- App.js
import Nav from './Nav';
function App() {
  return (
    <div className='App'>
      <h1>Routing Example with Best Practices</h1>
      <Nav />
      <Routes>
        <Route path='/about' element={<About />} />
        <Route path="/" exact="true" element={<Home />} /> //
exact="true" is function that run the default route.
      </Routes>
    </div>
  )
}

```

3. Component in Separate file.

```

Ans :- App.js
import Home from './Home';
import About from './About';

```

```

Home.js
export default function Home() {
  return (
    <div>
      <h1>Home</h1>
      <p>This is a Home page.</p>
    </div>
  )
}

```

```

About.js
export default function About() {
  return (
    <div>
      <h1>About</h1>
      <p>This is a About page</p>
    </div>
  )
}

```

4. Complete example.

```

Ans :- App.js
import { Routes, Route, Link } from 'react-router-dom';
import Nav from './Nav';
import Home from './Home';
import About from './About';
function App() {
  return (
    <div className='App'>
      <h1>Routing Example with Best Practics</h1>
      <Nav />
      <Routes>
        <Route path='/about' element={<About />} />
        <Route path='/' exact="true" element={<Home />} /> //
exact="true" is function that run the default route.
      </Routes>
    </div>
  )
}

```

5. Interview question.

Q :- Why used Link because we have already anchor tag?

Ans :- No, We don't used the anchor tag directly.

*/

/***** 404 Page - Page Not Found *****/

/*

1. Import React Router.

Ans :- import { Routes, Route, Link } from 'react-router-dom';

2. Make simple Routing.

```

Ans :- App.js
import PageNotFound from './PageNotFound';
function App() {
  return (
    <div className='App'>
      <h1>404 Page - Page Not Found</h1>
      <Nav />
      <Routes>

```

// When we used the Routes. With the Route we used Switch as a wrapper

```
<Route path='/about' element={<About />} />
<Route path="*" element={<PageNotFound />} /> // "*" is
mention to if route is not found then what route is open.
<Route path="/" exact="true" element={<Home />} /> //
exact="true" is function that run the default route.
```

```
</Routes>
</div>
)
}
```

3. Add 404 Page.

```
Ans :- PageNotFound.js
export default function PageNotFound(){
  return(
    <div>
      <span style={{color:"Red", fontSize:50}}>Page Not
Found</span>
    </div>
  )
}
```

4. Interview question.

Q :- Why we use the Routes tag?

Ans :-

*/

/***** Dynamic Routing With Params *****/

/*

1. What is Dynamic Routing.

Ans :- Dynamic Routing is a use When we have a unlimited routes and show according to it's name or id etc that dynamic routing.

For example :- Products inside the Shopping sites.

2. Make a list with user data.

Ans :- App.js

```
import UserDynamicRouting from './UserDynamicRouting';
let users = [
  { id:1, name:"Harshdeep Singh", email:"harsh@demo.com"},
  { id:2, name:"Prabhjot Singh", email:"prabh@demo.com"},
  { id:3, name:"Kirti", email:"kirti@demo.com"},
  { id:4, name:"Gurnoor Singh", email:"garry@demo.com"},
  { id:5, name:"Ankush", email:"ankush@demo.com"}
]
```

3. Add Routing.

Ans :- App.js

```
{
  users.map((item) =>{
    return (
      <div>
        //<h4>{item.name}</h4>
        <Link
to={'/user/'+item.id+'/'+item.name+'/'+item.email}><h4>{item.name}</h4></
Link>
      </div>
    )
  }
}
```

```

    }
    <Routes>
      <Route path='/user/:id/:name/:email'
element={<UserDynamicRouting />} />
    </Routes>

```

4. Make Dynamic Rounting.

Ans :- App.js

```

    <Routes>
      <Route path='/user/:id/:name/:email'
element={<UserDynamicRouting />} />
    </Routes>

```

```

UserDynamicRouting.js
import { useParams } from 'react-router-dom';
export default function UserDynamicRouting() {
  const {id,name,email} = useParams();
  //console.log(id);
  return(
    <>
      <h1>User Dynamic Routing</h1>
      <h4>Hi this is a user no :- {id}</h4>
      <h4>This is a user name :- {name}</h4>
      <h4>User email Id :- {email}</h4>
    </>
  )
}

```

```

Full Code App.js
import UserDynamicRouting from './UserDynamicRouting';
function App() {
  let users = [
    { id:1, name:"Harshdeep Singh", email:"harsh@demo.com"},
    { id:2, name:"Prabhjot Singh", email:"prabh@demo.com"},
    { id:3, name:"Kirti", email:"kirti@demo.com"},
    { id:4, name:"Gurnoor Singh", email:"garry@demo.com"},
    { id:5, name:"Ankush", email:"ankush@demo.com"}
  ]
  return (
    <div className='App'>
      <h1>Dynamic Routing With Params</h1>
      {
        users.map((item) =>{
          return (
            <div>
              // <h4>{item.name}</h4>
              <Link
to={'/user/'+item.id+'/'+item.name+'/'+item.email}><h4>{item.name}</h4></
Link>
            </div>
          )
        }
      )
    </div>
    <Routes>
      <Route path='/user/:id/:name/:email'
element={<UserDynamicRouting />} />
    </Routes>

```

```

    </div>
  )
}
5. Interview question.
*/

/***** Call Get Method API *****/
/*
1. What is API.
Ans :- Application Programming Interface.
2. Call API.
Ans :- When we call the API then we used the fetch() method. Inside this
put the api url. When we used this
API it's return the Promise. And return the Promise used the then()
method.
3. Get API data in State.
Ans :- function App() {
  // Don't use the API directly in the functional component. If you use
  in the class use the componentDidMount().
  //   fetch("http://localhost:5000/list").then((res) => {
  //     //res.json();
  //     console.log(res.json());
  //   }).catch((error) => {
  //     console.log(error);
  //   });
  const [data, setData]=useState([]);
  useEffect(() => {
    fetch("http://localhost:5000/list").then((result) => {
      result.json().then((resp) => {
        //console.log(resp);
        setData(resp)
      })
    }).catch((error) => {
      console.log(error);
    });
  }, []);
  // Pass the blank array for runing only one time [] in the useEffect.
  //console.log("Json data ===== ",data);

  return (
    <div className='App'>
      <Table striped hover variant='dark'>
        <thead>
          <tr>
            <th>Name</th>
            <th>Price</th>
            <th>Category</th>
            <th>Brand</th>
          </tr>
        </thead>
        <tbody>
          {
            data.map((item) =>
              <tr>
                <td>{item.name}</td>
                <td>{item.price}</td>
                <td>{item.category}</td>
                <td>{item.brand}</td>
              </tr>
            )
          }
        </tbody>
      </Table>
    </div>
  );
}

```

```

        </tr>
      )
    }
  </tbody>
</Table>
</div>
)
}
4. Make list from API.
Ans :- return (
  <div className='App'>
    <Table striped hover variant='dark'>
      <thead>
        <tr>
          <th>Name</th>
          <th>Price</th>
          <th>Category</th>
          <th>Brand</th>
        </tr>
      </thead>
      <tbody>
        {
          data.map((item) =>
            <tr>
              <td>{item.name}</td>
              <td>{item.price}</td>
              <td>{item.category}</td>
              <td>{item.brand}</td>
            </tr>
          )
        }
      </tbody>
    </Table>
  </div>
)
*/

```

```

/***** PostMan *****/
/*

```

1. What is Postman.
2. How to install it.
3. How to used it.
4. Test API.
5. Interview question.

Q :- Differnce between PUT and PATCH method?

Ans :- PUT :- Put method is used the send and modify the data.

PATCH :- PATCH method is just instruction to modify the data. And replace the original version of data.

There is no actual data.

```

*/

```

```

/***** POST method with API *****/

```

```

/*

```

1. Why we use the POST method.

Ans :- POST method is used to stored the data.

2. Make a form.

Ans :- return (

```

    <div className='App'>
      <h1>POST method with API</h1>
      <input type='text' name='name' value={name} onChange={
(e) => setName(e.target.value) }/><br /><br />
      <input type='price' name='price' value={price} onChange={
(e) => setPrice(e.target.value) }/><br /><br />
      <input type='brand' name='brand' value={brand} onChange={
(e) => setBrand(e.target.value) }/><br /><br />
      <button type='button' onClick={() => saveUser()}>Save new
user</button>
    </div>
  )

```

3. Submit data to API.

```

Ans :- fetch("http://localhost:5000/create",{
  method:"POST",
  headers:{
    "Accept":"application/json",
    "Content-Type":"application/json"
  },
  body:JSON.stringify(data)
}).then((result) => {
  //console.log(result);
  result.json().then((resp) => {
    console.log("Insert Data => ",resp);
  })
})

```

4. Test Result.

```

Ans :- App.js
function App() {
  const [name,setName]=useState(0);
  const [price,setPrice]=useState(0);
  const [brand,setBrand]=useState(0);

  function saveUser(){
    //console.log(name,price,brand);
    let data={name,price,brand};
    fetch("http://localhost:5000/create",{
      method:"POST",
      headers:{
        "Accept":"application/json",
        "Content-Type":"application/json"
      },
      body:JSON.stringify(data)
    }).then((result) => {
      //console.log(result);
      result.json().then((resp) => {
        console.log("Insert Data => ",resp);
      })
    })
  }

  return (
    <div className='App'>
      <h1>POST method with API</h1>
      <input type='text' name='name' value={name} onChange={
(e) => setName(e.target.value) }/><br /><br />
      <input type='price' name='price' value={price} onChange={
(e) => setPrice(e.target.value) }/><br /><br />

```

```

        <input type='brand' name='brand' value={brand} onChange={
(e) => setBrand(e.target.value) }/><br /><br />
        <button type='button' onClick={() => saveUser()}>Save new
user</button>
    </div>
  )
}

```

5. Interview question.

Q :- Can we send the body in GET method?

Ans :- No, We can't send the body in GET method.

*/

```

/***** Delete Method with API *****/
/*

```

1. Why we use the Delete method.

```

Ans :- function deleteUser(userId){
  //alert("User name = " + userId);
  fetch(`http://localhost:5000/delete/${userId}`, {
    method: "DELETE"
  }).then((result) => {
    result.json().then((resp) =>{
      alert(resp);
      getList()
    })
  })
}

```

2. Add delete button in the list.

```

Ans :- return (
  <div className='App'>
    <Table striped hover variant='dark'>
      <thead>
        <tr>
          <th>Name</th>
          <th>Price</th>
          <th>Category</th>
          <th>Brand</th>
          <th>Action</th>
        </tr>
      </thead>
      <tbody>
        {
          data.map((item) =>
            <tr>
              <td>{item.name}</td>
              <td>{item.price}</td>
              <td>{item.category}</td>
              <td>{item.brand}</td>
              <td><button
onClick={()=>deleteUser(item._id)}>Delete</button></td>
            </tr>
          )
        }
      </tbody>
    </Table>
  </div>
)

```

3. Make a function and call delete API.

Ans :-

```
function App() {
  const [data, setData]=useState([]);
  // Page refresh with this code
  //   useEffect(() => {
  //     fetch("http://localhost:5000/list").then((result) => {
  //       result.json().then((resp) => {
  //         //console.log(resp);
  //         setData(resp)
  //       })
  //     }).catch((error) => {
  //       console.log(error);
  //     });
  //   }, []);

  //   function deleteUser(userId){
  //     //alert("User name = " + userId);
  //     fetch(`http://localhost:5000/delete/${userId}`, {
  //       method:"DELETE"
  //     }).then((result) => {
  //       result.json().then((resp) =>{
  //         alert(resp);
  //       })
  //     })
  //   }

  // }
```

// Page not refresh with this code.

```
useEffect(() => {
  getList();
});
function getList(){
  fetch("http://localhost:5000/list").then((result) => {
    result.json().then((resp) => {
      //console.log(resp);
      setData(resp)
    })
  }).catch((error) => {
    console.log(error);
  });
}

function deleteUser(userId){
  //alert("User name = " + userId);
  fetch(`http://localhost:5000/delete/${userId}`, {
    method:"DELETE"
  }).then((result) => {
    result.json().then((resp) =>{
      alert(resp);
      getList()
    })
  })
}

}
return (
  <div className='App'>
    <Table striped hover variant='dark'>
```

```

        <thead>
          <tr>
            <th>Name</th>
            <th>Price</th>
            <th>Category</th>
            <th>Brand</th>
            <th>Action</th>
          </tr>
        </thead>
        <tbody>
          {
            data.map((item) =>
              <tr>
                <td>{item.name}</td>
                <td>{item.price}</td>
                <td>{item.category}</td>
                <td>{item.brand}</td>
                <td><button
onClick={ ()=>deleteUser(item._id)}>Delete</button></td>
              </tr>
            )
          }
        </tbody>
      </Table>
    </div>
  )
}

```

4. Interview question.
*/

/***** Pre-filled Form *****/

/*

1. Make a simple Form.

```

Ans :- <div>
      <input type='text' value={name} /><br /><br />
      <input type='text' value={price} /><br /><br />
      <input type='text' value={brand} /><br /><br />
      <Button>Update User</Button>

    </div>

```

2. Add update button in the list.

Ans :- <Button>Update User</Button>

3. Set value in List.

```

Ans :- function getList() {
  fetch("http://localhost:5000/list").then((result) => {
    result.json().then((resp) => {
      //console.log(resp);
      setUser(resp);

```

```

      // This is not working with mongodb in think is use the
      another data base like mysql in the video
      // setName(resp[0].name);
      // setPrice(resp[0].price);
      // setBrand(resp[0].brand);
      setName(resp.name);
      setPrice(resp.price);

```

```

        setBrand(resp.brand);
      })
    }).catch((error) => {
      console.log(error);
    });
  }
}

```

4. Populate value in form.

```

Ans :- function selectUser(id) {
  //console.log(users);
  // This is not working with mongodb in think is use the another
data base like mysql in the video
  // let userItem = users[id-1];
  // setName(userItem.name);
  // setPrice(userItem.price);
  // setBrand(userItem.brand);
  users.map((item) => {
    if(id === item._id) {
      //console.log(item);
      setName(item.name);
      setPrice(item.price);
      setBrand(item.brand);
    }
  });
}

```

App.js

```

function App() {
  const [users, setUser] = useState([]);
  const [name, setName] = useState(0);
  const [price, setPrice] = useState(0);
  const [brand, setBrand] = useState(0);

  useEffect(() => {
    getList();
  });

  function getList() {
    fetch("http://localhost:5000/list").then((result) => {
      result.json().then((resp) => {
        //console.log(resp);
        setUser(resp);

        // This is not working with mongodb in think is use the
another data base like mysql in the video
        // setName(resp[0].name);
        // setPrice(resp[0].price);
        // setBrand(resp[0].brand);
        setName(resp.name);
        setPrice(resp.price);
        setBrand(resp.brand);
      })
    }).catch((error) => {
      console.log(error);
    });
  }

  function deleteUser(userId) {
    //alert("User name = " + userId);
  }
}

```

```

        fetch(`http://localhost:5000/delete/${userId}`, {
            method: "DELETE"
        }).then((result) => {
            result.json().then((resp) => {
                alert(resp);
                getList()
            })
        })
    }

    function selectUser(id) {
        //console.log(users);
        // This is not working with mongodb in think is use the another
data base like mysql in the video
        // let userItem = users[id-1];
        // setName(userItem.name);
        // setPrice(userItem.price);
        // setBrand(userItem.brand);
        users.map((item) => {
            if(id === item._id) {
                //console.log(item);
                setName(item.name);
                setPrice(item.price);
                setBrand(item.brand);
            }
        });
    }

    return (
        <div className='App'>
            <Table striped hover variant='dark'>
                <thead>
                    <tr>
                        <th>Name</th>
                        <th>Price</th>
                        <th>Category</th>
                        <th>Brand</th>
                        <th>Action</th>
                    </tr>
                </thead>
                <tbody>
                    {
                        users.map((item) =>
                            <tr>
                                <td>{item.name}</td>
                                <td>{item.price}</td>
                                <td>{item.category}</td>
                                <td>{item.brand}</td>
                                <td>
                                    <button
onClick={ () => deleteUser(item._id) }>Delete</button>
                                    <button onClick={ () =>
selectUser(item._id) }>Update</button>
                                </td>
                            </tr>
                        )
                    }
                </tbody>
            </Table>
        </div>
    )

```

```

    </Table>
    <div>
      <input type='text' value={name} /><br /><br />
      <input type='text' value={price} /><br /><br />
      <input type='text' value={brand} /><br /><br />
      <Button>Update User</Button>

    </div>
  </div>
)
}

```

5. Interview question.

Q :- Can use the single state rather than use multiple state like that :-

```

const [name, setName] = useState(0);
const [price, setPrice] = useState(0);
const [brand, setBrand] = useState(0);

```

Ans :-

```

*/

/***** Update Data with PUT method *****/
/*
1. Recap Last video.
2. Update state for update data.
3. Send data with PUT API.
Ans :- function updateUser(){
  let item = {name,price,brand,userId};
  fetch(`http://localhost:5000/update/${userId}`, {
    method:"PUT",
    headers:{
      'Accept':'application/json',
      'content-type':'application/json'
    },
    body:JSON.stringify(item)
  }).then((result) => {
    result.json().then((resp) => {
      console.log(resp);
    })
  })
}

```

4. Test Result.

Ans :- Code is not working fine. But why that is.

```

function App() {
  const [users, setUser] = useState([]);
  const [name, setName] = useState("");
  const [price, setPrice] = useState("");
  const [brand, setBrand] = useState("");
  const [userId, setuserId] = useState(null);

  useEffect(() => {
    getList();
  });
  function getList(){
    fetch("http://localhost:5000/list").then((result) => {
      result.json().then((resp) => {
        //console.log(resp);
      })
    })
  }
}

```

```

        setUser(resp);

        // This is not working with mongodb in think is use the
another data base like mysql in the video
        // setName(resp[0].name);
        // setPrice(resp[0].price);
        // setBrand(resp[0].brand);
        setName(resp.name);
        setPrice(resp.price);
        setBrand(resp.brand);
        setUserId(resp._id);
    })
}).catch((error) => {
    console.log(error);
});
}

function deleteUser(userId) {
    //alert("User name = " + userId);
    fetch(`http://localhost:5000/delete/${userId}`, {
        method: "DELETE"
    }).then((result) => {
        result.json().then((resp) => {
            alert(resp);
            getList()
        })
    })
}

function selectUser(id) {
    //console.log(users);
    // This is not working with mongodb in think is use the another
data base like mysql in the video
    // let userItem = users[id-1];
    // setName(userItem.name);
    // setPrice(userItem.price);
    // setBrand(userItem.brand);
    users.map((item) => {
        if(id === item._id) {
            //console.log(item);
            setName(item.name);
            setPrice(item.price);
            setBrand(item.brand);
            setUserId(item._id);
        }
    });
}

function updateUser() {
    let item = {name, price, brand, userId};
    fetch(`http://localhost:5000/update/${userId}`, {
        method: "PUT",
        headers: {
            'Accept': 'application/json',
            'content-type': 'application/json'
        },
        body: JSON.stringify(item)
    }).then((result) => {

```

```

        result.json().then((resp) => {
            console.log(resp);
        })
    })
}
return (
    <div className='App'>
        <Table striped hover variant='dark'>
            <thead>
                <tr>
                    <th>Name</th>
                    <th>Price</th>
                    <th>Category</th>
                    <th>Brand</th>
                    <th>Action</th>
                </tr>
            </thead>
            <tbody>
                {
                    users.map((item) =>
                        <tr>
                            <td>{item.name}</td>
                            <td>{item.price}</td>
                            <td>{item.category}</td>
                            <td>{item.brand}</td>
                            <td>
                                <button
onClick={ () => deleteUser(item._id) }>Delete</button>
                                <button onClick={ () =>
selectUser(item._id) }>Update</button>
                            </td>
                        </tr>
                    )
                }
            </tbody>
        </Table>
        <div>
            <input type='text' value={name} onChange={(e) =>
setName(e.target.value) } /><br /><br />
            <input type='text' value={price} onChange={(e) =>
setPrice(e.target.value) } /><br /><br />
            <input type='text' value={brand} onChange={(e) =>
setBrand(e.target.value) } /><br /><br />
            <Button onClick={updateUser}>Update User</Button>
        </div>
    </div>
)
}

```

5. Interview question.

*/

/* ***** Previous State in functional component ***** */

1. What is previous state.

Ans :- Use the last state is Previous state.

2. Why we need it.

3. How to get it and used it.

```

Ans :- function App() {
  const [count, setCount]=useState(1);
  function updateCounter(){
    // First method
    // //setCount(count+1)
    // let ramValue = Math.floor(Math.random()*10);
    // //setCount(ramValue);
    // setCount((preValue) => {
    //   console.log(preValue);
    //   // if(preValue - ramValue < 5){
    //     alert("Prvious Value")
    //   }
    //   if(preValue < 5){
    //     alert("Low value");
    //   }
    //   return ramValue;
    // });
    // Second Method
    for(let i=0;i<5;i++){
      setCount((preValue) => {
        return preValue+1
      })
    }
  }
  return (
    <div className='App'>
      <h1>Previous State in functional component</h1>
      <h5>{count}</h5>
      <Button onClick={updateCounter}>Update Counter</Button>
    </div>
  )
}

```

4. Another important and trick example.

5. Interview question.

*/

```

/***** Previous Props with Hooks *****/
/*

```

1. What is Previous Props.

Ans :- Suppose we have a counter and onclick we increase the value. The last value is 4. So how to get the previous value.

2. Why we need it.

Ans :- Sometime we performe some logic and operation to check the difference between last and current value.

This is a advanced concept because when we working on advance project then we used it.

3. How to used it and get it.

Ans :-

```

import PreviousPropsWithHook from './PreviousPropsWithHook';
function App() {
  const [count, setCount] = useState(0);
  return (
    <div className='App'>
      <h1>Previous Props with Hook</h1>
      <PreviousPropsWithHook count={count} />
      <Button onClick={() => setCount(Math.floor(Math.random() *
10))}>Update Counter</Button>
    </div>
  )
}

```

```
)
}
```

4. We will use useRef hook for it.

```
Ans :- App.js
import PreviousPropsWithHook from './PreviousPropsWithHook';
function App() {
  const [count, setCount] = useState(0);
  return (
    <div className='App'>
      <h1>Previous Props with Hook</h1>
      <PreviousPropsWithHook count={count} />
      <Button onClick={() => setCount(Math.floor(Math.random() *
10))}>Update Counter</Button>
    </div>
  )
}
```

```
PreviousPropsWithHook.js
import React, {useEffect, useRef} from 'react';
export default function PreviousPropsWithHook(props){
  const lastValue = useRef();
  useEffect(() => {
    lastValue.current=props.count;
  });
  const previousValue = lastValue.current;
  return(
    <>
      <h2>PreviousPropsWithHook Component</h2>
      <h6>Current Value {props.count}</h6>
      <h6>Previous value {previousValue}</h6>
      <h6>Differnce between value {props.count -
previousValue}</h6>
    </>
  );
}
```

5. Interview question.

*/

```
/***** Context API *****/
/*
```

1. What we are going to do.

Ans :- We pass the data into one component to another component easily or direct.

2. Make common file for context file.

```
Ans :- CommonContext.js
import React from "react";
export const CommonContext = React.createContext();
```

3. Make provider wrapper in app.js.

Ans :- Provider is set the data.

```
App.js
import { CommonContext } from './components/CommonContext';
import { render } from '@testing-library/react';
import Main from './components/Main';
import UpdateButton from './components/UpdateButton';
import Header from './components/Header';
import Footer from './components/Footer';
```

```

class App extends React.Component{
  constructor(){
    super();
    // This is a function.
    this.updateColor=(color) => {
      // this.setState({
      //   color:"red"
      // })
      // Pass dynamic color
      this.setState({
        color:color
      })
    }
    this.state = {
      color:"green",
      updateColor:this.updateColor
    }
  }
  render() {
    return (
      <div className='App'>
        <CommonContext.Provider value={this.state}>
          <Header />
          <h1>Context API</h1>
          <Main />
          <UpdateButton />
          <Footer />
        </CommonContext.Provider>
      </div>
    )
  }
}

```

4. Make main js file for consumer context.

Ans :- Consumer mean's used the data.

```

Main.js
import React from 'react';
import { CommonContext } from './CommonContext';
export default class Main extends React.Component{
  render(){
    return(
      <div className="App">
        <CommonContext.Consumer >
          {
            ({color})=>(
              <h1 style={{backgroundColor:color}}>Hello
this is Main Page.</h1>
            )
          }
        </CommonContext.Consumer>
      </div>
    )
  }
}

```

5. Make Button file for update context.

Ans :- UpdateButton.js

```

import React from 'react';

```

```

import { CommonContext } from './CommonContext';
import { Button } from 'react-bootstrap';
export default function UpdateButton() {
  return (
    <CommonContext.Consumer>
    {
      ({updateColor}) => (
        <Button onClick={() =>
updateColor('orange')}>Update Color</Button>
        )
      }
    </CommonContext.Consumer>
  )
}

```

6. Make Header and Footer file.

Ans :- Header.js

```

import React from 'react';
import { CommonContext } from './CommonContext';
export default class Header extends React.Component{
  render(){
    return(
      <div className="App">
        <CommonContext.Consumer >
          {
            ({color})=>(
              <h1
style={{backgroundColor:color}}>Header Page.</h1>
              )
            }
          </CommonContext.Consumer>
        </div>
      )
    }
  }
}

```

Footer.js

```

import React from 'react';
import { CommonContext } from './CommonContext';
export default class Footer extends React.Component{
  render(){
    return(
      <div className="App">
        <CommonContext.Consumer >
          {
            ({color})=>(
              <h1
style={{backgroundColor:color}}>Footer Page.</h1>
              )
            }
          </CommonContext.Consumer>
        </div>
      )
    }
  }
}
*/

```

/***** Basic Interview Question ReactJs *****/

/*

1. What is ReactJs.

Ans :- React is a javascript library.
Use for fronted development with good speed.
Developed by Facebook.
The current version is 18.0.0

2. What is component.

Ans :- A piece of code that can reuse like function. But more powerfull than function in this we have use HTML, API, Function calling etc. 2 types of component 1. Class and functional component.

3. What is class component.

Ans :- A component that has ES6 class. More complex and more powerfull than component. Because it has a life cycle method and state. No need to use Hooks.

4. What is Functional component.

Ans :- A component that has no ES6 class. It uses a simple function. Simple and need hooks for performing state and lifecycle method. Simple to define and recommaned.

5. What is State.

Ans :- Information holder inside a component and it store any data like a variable. When we update state component will rerender again but no in case of variable. We can not use State outside to component.

5.1 Can we used the one component to any where.

Ans :- No

6. What is Props.

Ans :- Share data between component. We can send the data parent to child, Child to parent and in sibling component. We can not change props in receiving component. We can not update from receiving side.

7. What is Life Cycle method.

Ans :- Some method will execute on the component mount, Update and Unmount we call them life cycle method.

Example :- 1. componentDidMount
2. componentDidUpdate
3. shouldComponentUpdate

8. What are the Hooks.

Ans :- Functional component do not have state and life cycle method. To use life cycle method and state we can import hooks in functional components. We can't use hooks in class component.

9. What are the useState.

Ans :- This is a Hook for use state in functional component.

10. What is useEffect.

Ans :- This is a hook for use lifecycle methods in a functional component.

11. What is NPM.

Ans :- This is Node Package Manager which help us to install library in ReactJs.

Example :- Map, Form validation etc.

12. What is Package.js.

Ans :- This file profile application details such as version, node package, description etc.

13. Can we use ReactJs without npm.

Ans :- Yes, We can use it like jquery libaray also. for that we can in root file aur use cdn. But this is not recommended. Because we can't install packages and file is not minified.

14. What we need to Develop React App.

Ans :- For the recommended way we need npm, node, and any code editor. Otherwise we can use CDN also.

15. Can we use TS with ReactJs.

Ans :- Yes, we can. You need to compiler for this.

16. Can we use Angular, JQuery, and Bootstrap with ReactJs.

Ans :- Yes, we can. We can install package for JQuery and Bootstrap. We can use React package inside Angular with npm install.

17. Can we connect ReactJs with Database directly.

Ans :- No, We need to API for DB data.

18. What is SPA.

Ans :- It stand for single page application. Which mean's application look like a single page and No reload or refresh on jumping on other page and link. We can make single page application with React, Angular and Vue.

19. What is Jest.

Ans :- Jest is a Unit Testing Framwork. Jest is recommended and easy to use with React.

*/